Démonstrateur d'attaques contre un ML-NIDS

Du réseau dessiné à la détection en direct

Projet VAP SSR 2026–2027

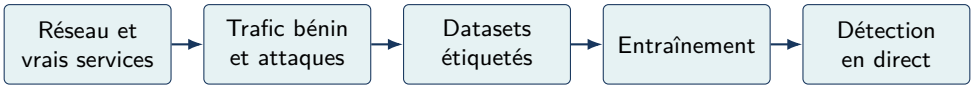
Réunion avec Grégory Blanc

9 novembre 2026

Avancées depuis la réunion du 27 octobre

Le 27 octobre : topologie, conteneurs, capture. La détection ML et les attaques restaient à intégrer.

Aujourd'hui, la chaîne existe de bout en bout dans l'interface :



Ce qui reste à faire

Le cœur du sujet : **les attaques contre le modèle** (évasion, empoisonnement).

Les serveurs font tourner de vrais services

Pourquoi : une simple bannière ne produit pas les échanges d'une vraie session. Le modèle apprendrait des artefacts du lab.

Service	**Ce qui tourne**
HTTP, HTTPS	serveur web Python : site de 20 pages, téléchargements, API JSON, connexion ; TLS 1.3
SSH, FTP, Telnet	OpenSSH (avec SFTP), vsftpd, telnetd
SMTP, DNS, MySQL	OpenSMTPD, dnsmasq (noms en `.lab`), MariaDB (base « boutique »)
Autre nom (ex. pop3)	**simulé** : bannière plausible en TCP, écho en UDP

Le type de service est reconnu par son **nom** ; le port usuel ne sert que s'il n'y a pas de nom.

Le modèle : un random forest entraîné sur notre réseau

- **Aucun modèle préentraîné :** il est entraîné sur les datasets générés dans le lab.
- **Entrée :** des flux décrits par environ 80 features CICFlowMeter.
- **Réglages :** 100 arbres par défaut, classes binaires ou multiclasses, `class_weight="balanced"`.
- **Features retirées :** identifiants, adresses IP, ports. Sinon le modèle apprendrait « PC-1 attaque » plutôt qu'un comportement.
- **Test :** sur des variantes séparées (autres machines, vitesses, intensité du bénin). Pas de découpage aléatoire, qui gonflerait les scores.

Rapport : attaques détectées, fausses alertes, F1 macro, matrice de confusion, features les plus utiles.

ConCap : ce qu'on a repris, ce qu'on a remplacé

Repris

- La méthode : scénario, capture, CICFlowMeter, étiquetage connu d'avance.
- L'**image CICFlowMeter conteneurisée**, utilisée telle quelle.
- Des noms de classes proches des jeux CIC.

Remplacé par le moteur du lab

- L'orchestration (Kubernetes $\rightarrow$ Docker).
- Le point de capture (cible $\rightarrow$ sonde).
- L'étiquetage et le découpage des flux.

Mêmes colonnes que ConCap et CIC-IDS : **les données restent comparables**.

Les attaques disponibles : nmap et hydra

Scénario	**Outil**	**Classe**
Balayage de découverte	`nmap -sn`	ping-sweep
Scan de ports (connect, 200 ports)	`nmap -sT`	port-scan
Scan furtif (SYN, tous les ports)	`nmap -sS`	syn-scan
Identification des services	`nmap -sV`	service-detection
Scan approfondi	`nmap -A`	aggressive-scan
Force brute SSH	hydra	ssh-patator
Force brute FTP	hydra	ftp-patator

Quatre vitesses, depuis n'importe quel équipement. Couverture limitée à la **reconnaissance** et à la **force brute** : pas de DoS, ni d'attaque web, ni d'exploitation.

Pourquoi ne pas utiliser ConCap tel quel ?

	ConCap	**Besoin du projet**
Infrastructure	cluster Kubernetes	Docker seul, piloté depuis l'interface
Topologie	réseau plat, sans routeur	LAN, DMZ, routeurs
Capture	sur la cible	sur la sonde IDS
Trafic bénin	pas de trafic de fond	continu, pendant les attaques
Étiquetage	une étiquette par scénario	par flux, dans un trafic mélangé
Flux	pcap entier	fenêtres de 5 s, comme la détection

L'argument décisif : le point de vue

Un modèle apprend les flux tels qu'ils sont vus à l'endroit de la capture. Entraîné sur la vue d'une cible isolée, il serait évalué sur des flux qui ne ressemblent pas à ce qu'il a appris.

Pourquoi entraîner plutôt que prendre un modèle préentraîné ?

1. **Il faudrait quand même nos données.** Pour l'adapter à notre réseau, la génération de datasets reste nécessaire.
2. **Un modèle appris ailleurs apprend aussi cet ailleurs :** latences, services, trafic normal. Notre lab est différent.
3. **Entraîner n'est pas le coût principal.** C'est la production de données étiquetées qui l'est.
4. **Le fine-tuning s'applique mal à un random forest :** pas de poids à réajuster, les anciens arbres continuent de voter.
5. **Le sujet demande la maîtrise du modèle.** L'évasion s'appuie sur ce qu'il a appris, l'empoisonnement sur ses données.

Piste : un random forest entraîné sur CIC-IDS2017, testé sur notre lab, comme point de comparaison externe.

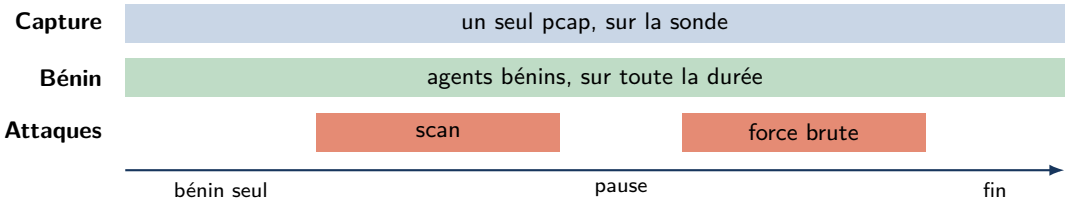
Construire un dataset : qui fait quoi ?

Le trafic n'est pas simulé : seule la décision est scriptée, les paquets sont réels.

Rôle	Implémentation	Ce qu'il fait
Chef d'orchestre	moteur Python	démarre et arrête tout, note les horaires de chaque attaque
Trafic bénin	agent Python, un par émetteur	choisit des sessions et lance de vrais clients : navigation web, SSH, FTP, mail, MySQL, DNS, ping
Attaques	nmap et hydra	les vrais outils, lancés par le moteur
Capture	tcpdump sur la sonde	enregistre un pcap unique, depuis le port miroir

L'agent partage la pile réseau de l'émetteur : pour la sonde, c'est bien le PC qui parle.

Bénin et attaques sont joués en même temps



- Les attaques passent **l'une après l'autre**, au milieu du trafic normal.
- L'agent bénin de la **machine attaquante** est mis en pause pendant l'attaque : tout ce qui va de la source vers la cible est donc bien de l'attaque.
- **Résultat :** pcap, CSV étiqueté, métadonnées ; puis 2 variantes rejouées pour le test.

La fenêtre de 5 secondes

Le pcap est découpé après la capture, et CICFlowMeter traite chaque tranche.

- La détection en direct analyse le trafic par tranches de 5 s : elle n'attend pas la fin des conversations.
- L'entraînement doit voir les flux **de la même façon**, sinon le modèle ne les reconnaît pas.
- Limite : une conversation longue est coupée (SSH de 12 s $\approx$ 3 flux).

À ne pas confondre	**Fenêtre de 5 s**	**Fenêtre d'attaque**
Durée	fixe	celle de l'attaque, -1 s et $+2$ s de marge
Rôle	découper les flux	attribuer l'étiquette

Durée et composition d'un dataset

Rien de fixe : on compose le programme ligne par ligne (attaque, source, cible, vitesse, répétitions).

Paramètre	**Défaut**
Bénin seul au début	30 s
Bénin entre deux attaques	10 s
Bénin seul à la fin	20 s
Durée maximale d'une attaque	60 s
Répétitions par ligne (max. 10)	2
Variantes de test	2

Exemple : scan + force brute

2 lignes, 2 répétitions, soit 4 attaques :
$30 + 4 \times 60 + 3 \times 10 + 20 = 320$ s au plus.

Soit **environ 16 min au plus** pour les 3 datasets, hors extraction des flux.

Point d'attention : un scan produit beaucoup de flux, une force brute très peu (8 mots de passe essayés).

Démonstration

Générer, entraîner, détecter, attaquer

Notre dataset est-il trop pauvre ?

Quantité : plutôt suffisante

- Un random forest se contente de peu de données tabulaires.
- À surveiller : les classes de force brute, peu nombreuses.
- `class_weight` compense le déséquilibre, pas le manque de diversité.

Diversité : le vrai point faible

- 7 scénarios, deux familles d'attaques.
- Un seul compte, un seul site, pas de profils d'utilisateurs.
- Une topologie, une latence quasi nulle.

Conséquence : de très bons scores prouveraient peu. Séparer un scan de la navigation web est facile, et le modèle peut s'appuyer sur des artefacts du lab.

Est-ce gênant pour le projet ?

Pas pour l'objectif : montrer le principe d'une attaque, en action, dans le lab.

Exemple visé

Modifier les timings : l'IDS ne détecte plus, **et l'attaque fonctionne toujours**.

Mais deux conditions pour que la démonstration ait du sens :

1. **Une ligne de base solide :** le modèle doit bien détecter l'attaque non modifiée, y compris sur les variantes.
2. **Attribuer la cause :** ralentir une attaque change aussi le découpage en fenêtres de 5 s. La détection pourrait chuter à cause du découpage, pas du modèle. *(Hypothèse à tester en faisant varier la fenêtre.)*

La suite : plusieurs modèles, réentraînés sur nos données

Le sujet demande « plusieurs attaques » contre « plusieurs » modèles. Ce qu'on reprend de la littérature, c'est l'**architecture**, pas les poids.

Priorité	**Modèle**	**Intérêt**
Fait	Random forest	cible de base (Das)
1	MLP ou XGBoost	mêmes CSV, peu de travail ; test du **transfert** d'une attaque (Das, Ma, Okada)
2	Autoencodeur sur flux	non supervisé, entraîné sur le bénin seul (Sharon)
Si possible	KitNET, GNN	pipeline différent : paquets (Han) ou graphe (Venturi)

Notre positionnement : une preuve de concept mesurée

On ne construit pas le meilleur IDS : le sujet demande de *caractériser le réalisme* des scénarios d'attaque.

Quatre mesures pour chaque attaque :

1. **Détection** avant et après l'attaque.
2. **Efficacité conservée :** hydra trouve-t-il encore le mot de passe, nmap les ports ?
3. **Coût** pour l'attaquant : temps perdu, trafic ajouté.
4. **Hypothèses :** ce que l'attaquant doit savoir ou contrôler (boîte blanche ou noire, accès aux données d'entraînement).

Limite assumée : une évasion réussie ici ne prouve pas qu'elle marcherait telle quelle sur un IDS de production.

Prochaines étapes

1. **Établir la ligne de base :** générer des datasets équilibrés et mesurer le modèle sur les variantes.
2. **Évasion par timing** sur le random forest, avec les quatre mesures.
3. **Deuxième modèle** (MLP ou XGBoost) et test de transfert.
4. **Empoisonnement ou porte dérobée :** injecter dans un dataset, puis jouer le déclencheur en direct (PCAP-Backdoor, Ma).

En option : comparaison avec un modèle entraîné sur CIC-IDS2017.

Questions pour Grégory Blanc

1. **Modèles :** combien en faut-il, et lesquels vous semblent prioritaires ?
2. **Réalisme :** les quatre mesures proposées correspondent-elles à ce que vous attendez ?
3. **Validité :** le lab seul suffit-il, ou faut-il une référence externe comme CIC-IDS2017 ?
4. **Attaques :** commencer par l'évasion par timing vous convient-il ? Quelle attaque d'empoisonnement privilégier ?